Kernel: sagemath

Práctica 1: revisión de SageMath

É necesario usar un Jupyter Notebook con Kernel SageMath,

1. ou ben en [cocalc.ai](),
2. ou ben nunha instalación local de SageMath. Para Windows recoméndase instalar a versión 9.1 ([SageMath-9.1-Installer-v0.6.0.exe]()).

Empezando con Sage

Unha variable `a` (ou *obxecto*) pode ser dunha *clase*, por exemplo `Rational`, e ter un valor (ou *atributo*), 1/3. As funcións (ou *métodos*) da clase como `parent` permiten interaccionar co obxecto.

In [1]:
```
a = 1/3
parent(a)
```

Out[1]:
```
Rational Field
```

Se non se indica o contrario, SageMath traballa con representación exacta (cálculo simbólico).

In [3]:
```
a = sin(1/3)
a, parent(a)
```

Out[3]:
```
(sin(1/3), Symbolic Ring)
```

En cálculo numérico traballaremos con números máquina do universo RDF (reais de punto flotante con 53 bits de precisión para a mantisa).

In [24]:
```
a = RDF(1/3)
a, parent(a)
```

Out[24]:
```
(0.3333333333333333, Real Double Field)
```

Un exemplo máis elaborado que pasa do expresión simbólica `pi/2` ao número máquina `RDF(pi/2)` e á cadea `str(RDF(pi/2))`. Obsérvese que `+` actúa de xeito distinto entre cadeas.

In [3]:
```
print('O resultado é ' + str(RDF(pi/2)))
```

Out[3]:
```
O resultado é 1.5707963267948966
```

O método `epsilon` de `RDF` devolve o valor de ε.

In [8]:
```
RDF.epsilon()
```

Out[8]:
```
2.220446049250313e-16
```

Expresións simbólicas en Sage

Ao inicio dunha sesión de Sage, `x` xa é unha expresión simbólica. Calquer outra variable débese converter explícitamente.

In [4]:
```
parent(x)
```

Out[4]:
```
Symbolic Ring
```

In [5]:
```
var('t')
parent(t)
```

Out[5]:
```
Symbolic Ring
```

O resto de expresións simbólicas se constrúen a partir destas:

In [26]:
```
f(x) = sin(x/2)
parent(f)
```

Out[26]:
```
Callable function ring with argument x
```

As expresións simbólicas permiten facer cálculos simbólicos.

In [8]:
```
limit(f, x=0)
```

Out[8]:
```
x |--> 0
```

In [9]:
```
show(diff(f,x))
```

Out[9]:

$$x \mapsto 21 \cos\left(21\, x\right)$$

A expresión anterior pódese gardar e usar como imaxe.

O método `save` permite facer esto automáticamente desde o código.

In [28]:
```
g = plot(f(x), 0, 1)
g.save('grafica.png')
```

Arrays en SageMath

Python permite usar listas (obxectos mutables).

In [11]:
```
l = [1,2,3]
l.append(4)
l
```

Out[11]:
```
[1, 2, 3, 4]
```

As listas non son vectores.

In [9]:
```
2*l
```

Out[9]:
```
[1, 2, 3, 4, 1, 2, 3, 4]
```

Para elo temos a clase `vector` de SageMath.

In [10]:
```
v = vector(RDF, [1,2,3,4])
2*v
```

Out[10]:
```
(2.0, 4.0, 6.0, 8.0)
```

O operador `[primero,segundo..ultimo]` permite xerar listas de xeito sinxelo.

In [28]:
```
[1..9]
```

Out[28]:
```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In [29]:
```
[1,3..9]
```

Out[29]:
```
[1, 3, 5, 7, 9]
```

O método `range(n)` é o preferido cando se debe iterar nos índices dunha lista ou vector, pois xera a lista `[0..n-1]`.

In [26]:
```
[1/v[i] for i in range(len(v))]
```

Out[26]:
```
[1.0, 0.5, 0.3333333333333333, 0.25]
```

Python permite listas de listas.

In [21]:
```
L = [[1,2],[2,3]]
L[0][0]
```

Out[21]:
```
1
```

Pero as matrices obtéñense coa clase `matrix` de SageMath. Obsérvese a notación `[fila,columna]` neste caso.

In [20]:
```
A = matrix(RDF,[[1,2],[2,3]])
# equivalentemente, A = matrix(RDF, 2, 2, [1,2,2,3])
A[0,0]
```

Out[20]:
```
1.0
```

Métodos para matrices (non dispoñibles para listas de listas).

In [22]:
```
A.nrows(), A.ncols(), A.rank(), A.det(), A.transpose(), A.inverse() #etc.
```

Out[22]:
```
(
                [1.0 2.0]  [-3.0  2.0]
2, 2, 2, -1.0, [2.0 3.0], [ 2.0 -1.0]
)
```

Hai máis formas de crear matrices.

In [23]:
```
matrix(RDF, 2), matrix.ones(RDF, 2, 3), matrix.identity(RDF, 2),
matrix.random(RDF,2)
```

Out[23]:
```
(
[0.0 0.0]  [1.0 1.0 1.0]  [1.0 0.0]
[0.0 0.0], [1.0 1.0 1.0], [0.0 1.0],

[-0.7717414427312617 0.31760795074114956]
[-0.8834913783901928  0.4665136884448533]
)
```

Ou dando unha regra.

In [24]:
```
A = matrix(RDF, 2, 2, lambda i,j: i+j+1)
A
```

Out[24]:
```
[1.0 2.0]
[2.0 3.0]
```

O método `apply_map` permite aplicar unha función a cada compoñente dun vector ou matriz.

In [35]:
```
f(x) = sin(x/2)
A.apply_map(f)
```

Out[35]:
```
[ 0.479425538604203 0.8414709848078965]
[0.8414709848078965 0.9974949866040544]
```

O método `map` fai o equivalente para listas Python.

In [39]:
```
l = [1..4]
list(map(f,l))
```

Out[39]:
```
[sin(1/2), sin(1), sin(3/2), sin(2)]
```

O método `.solve_right()` implementa o método de descomposición de Gauss con pivote.

In [5]:
```
A = matrix(RDF, [[1,2],[2,3]])
b = vector(RDF,[1,5])
A.solve_right(b) #solución de A*x=b
```

Out[5]:
```
(7.000000000000001, -3.0000000000000004)
```

Programación en Sage

As estructuras `if` permiten decidir en función dunha ou varias condicións:

In [15]:
```
i = 0 # asignar a i un valor enteiro
if i>0:
    print('positivo')
elif i<0:
    print('negativo')
else:
    print('cero')
```

Out[15]:
```
cero
```

As funcións `def` permiten implementar un *algoritmo*.

In [31]:
```
def f(x):
    if x >= 1:
        return x-1
    else:
        return 1-x
```

Obsérvese que, salvo casos sinxelos equiparables a unha expresión simbólica, xa non é posible facer cálculo simbólico.

In [32]:
```
diff(f(x)) # x >= 1 é sempre falso!
```

Out[32]:
```
-1
```

As funcións `lambda` son funcións que se poden escribir nunha liña e poden ser usadas como argumento doutras funcións ou métodos.

In [16]:
```
h = lambda i,j: i+j+1
matrix(RDF, 2, 2, h)
```

Out[16]:
```
[1.0 2.0]
[2.0 3.0]
```

Para percorrer listas basta visitar os seus valores:

In [17]:
```
v = [10,20,30]
suma = 0
for vi in v:
    suma += vi
suma
```

Out[17]:
```
60
```

Normalmente, podemos escribilo de xeito máis conciso. Por exemplo, como usar `for` para a suma ∑n=1100 n41:

In [33]:
```
sum(1/RDF(n)^4 for n in [1..100])
```

Out[33]:
```
1.0823229053444727
```

Se se debe traballar con varias listas, convén usar índices:

In [20]:
```
v = [10, 20, 30]
w = [40, 50, 60]
z = []
for i in range(len(v)):
   z.append(v[i]/w[i])
z
```

Out[20]:
```
[1/4, 2/5, 1/2]
```

E tamén podemos escribilo de xeito conciso accedendo aos puntos `(v,w)` xerados con `zip`:

In [21]:
```
z = [vi/wi for vi,wi in zip(v,w)]
z
```

Out[21]:
```
[1/4, 2/5, 1/2]
```

Se non coñecemos o número total de iteración, convén usar `while`. Por exemplo, para aproximar a suma ∑n=1∞ n41 cos termos menores que unha tolerancia `tol`.

In [23]:
```
def serie(tol):
    suma = RDF(0)
    n = 1
    term = RDF(1)
    while term > tol:
```

```
        suma += term
        n += 1
        term = 1/RDF(n)^4
    return suma

serie(1e-14)
```

Out[23]:
```
1.0823232337006023
```

Para consultar os distintos tipos de estructuras de decisión e bucles, consulta a web fundmat.

In [ ]:
```

```